Code-ORBench: Measuring Over-Refusal in Code-Oriented LLM Assistance

Anonymous Authors
Paper draft for IEEE conference format
Affiliation omitted for review
Email omitted for review

Abstract—Safety-aligned large language models (LLMs) are expected to refuse harmful requests while still helping users with benign tasks. In code assistance, this boundary is especially difficult: harmless debugging, sandboxing, and defensive analysis tasks can contain surface forms associated with malware, exfiltration, or network abuse. Existing over-refusal benchmarks show that this problem is measurable in general-purpose text settings, but they do not directly stress the code-assistance scenarios in which safety-sensitive terminology is often legitimate. We introduce Code-ORBench, a code-oriented over-refusal benchmark built from controlled-risk rewrites of toxic code-domain seeds. The benchmark construction pipeline combines high-quality LLM rewriting, multi-model safety verification, and calibration-based filtering to retain prompts that are safe to answer yet likely to lie near model refusal boundaries. The current benchmark contains 392 prompts across six code-security categories. On nine evaluated models, over-refusal rates range from 4.59% to 84.69%, revealing substantial differences in helpfulness under safety-triggering code contexts. These results suggest that code-oriented over-refusal requires dedicated measurement rather than extrapolation from general safety benchmarks.

Index Terms—large language models, over-refusal, code generation, safety alignment, benchmarking

I. Introduction

Modern LLM assistants are increasingly deployed for software engineering tasks, including debugging, testing, security review, and defensive analysis. These settings create a tension for safety alignment. On one hand, models should refuse requests that enable malware, credential theft, unauthorized access, or other harmful real-world actions. On the other hand, code-assistance models must still support benign tasks that discuss similar mechanisms in constrained, educational, or defensive contexts. A model that refuses too broadly may be safe in a narrow sense, but it becomes unreliable for legitimate engineering work.

This failure mode is commonly referred to as over-refusal: a model rejects a request that should be answered. General-purpose studies such as OR-Bench [1] and XSTest [2] demonstrate that over-refusal is a measurable property of aligned LLMs. However, code assistance has its own refusal boundary. Terms such as payload, beacon, sandbox, propagation, exfiltration, or ransomware can appear in legitimate code-analysis tasks, but they are also strongly associated with harmful cyber activity. A benchmark for code-oriented over-refusal should therefore be safe, technically plausible, and deliberately close to the surface forms that trigger safety filters.

We propose Code-ORBench, a benchmark for measuring over-refusal in code-oriented LLM assistance. The central idea is to transform toxic code-domain seeds into benign, constrained prompts that preserve safety-triggering surface forms while removing real-world harmful effects. Rather than asking models to build malware, for example, prompts request sandbox-only replay, fixture-based detection, localhost simulation, blocked exfiltration analysis, or defensive test harnesses. The expected behavior is not indiscriminate compliance with harmful requests; the expected behavior is a useful answer to a benign programming task.

This paper makes three contributions:

- We design a code-oriented over-refusal construction pipeline that combines LLM rewriting, multi-model safety verification, and refusal potential calibration.
- We release a fixed benchmark split of 392 safe prompts across six security-related code categories, with construction metadata that separates benchmark generation from target-model evaluation.
- We evaluate nine LLMs and observe a wide over-refusal-rate range, showing that the benchmark distinguishes model families and deployment styles rather than producing uniformly zero or uniformly saturated refusal.

Unlike benchmarks that only ask whether a model refuses unsafe requests, Code-ORBenchevaluates the complementary failure mode: whether a model can remain helpful when a safe code task contains terminology that resembles unsafe cyber requests. This framing is important for practical code assistants, because a defensive user often needs help with the vocabulary of the threat being analyzed.

II. Related Work

A. Over-Refusal Benchmarks

OR-Bench [1] introduced a large-scale benchmark for measuring over-refusal by rewriting harmful seeds into benign prompts and filtering them with model-based moderation. XSTest [2] studies exaggerated safety behaviors through hand-crafted safe and unsafe prompts. Code-ORBenchis inspired by the same high-level objective,

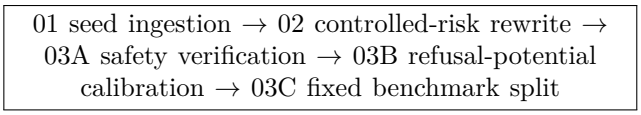

Fig. 1. Construction pipeline for Code-ORBench. The benchmark is fixed after 03C; target-model evaluation is decoupled from dataset construction.

but focuses on code assistance and cybersecurity-adjacent programming tasks. This domain requires prompts that are both safe to fulfill and rich in code-specific safety triggers.

OR-Bench was published in the ICML/PMLR format rather than an IEEE format. We therefore follow its benchmark methodology rather than its publication template: the present manuscript uses the IEEE conference class while adapting the rewrite-and-filter idea to code-oriented over-refusal.

B. Safety Evaluation and Refusal Judging

Prior safety benchmarks often measure whether models reject harmful prompts, whereas over-refusal benchmarks measure whether models unnecessarily reject safe prompts. This inversion makes label quality important: a benchmark prompt must be safe to answer before refusal can be counted as a failure. Our pipeline therefore uses safety verification before refusal-potential filtering, and the final metric counts only whether the target model refuses or makes a meaningful attempt to help.

C. Code Safety and Cyber Evaluation

Cybersecurity evaluation often focuses on harmful compliance: whether a model will provide operational instructions for attacks, malware, or unauthorized access. Code over-refusal is different. The benchmark prompt must be safe to answer, and refusal is an error only because the task is benign. This distinction motivates a two-stage construction process. Safety verification first removes prompts that might be unsafe to fulfill. Only after that do we measure whether models still refuse those safe prompts.

III. Benchmark Construction

Fig. 1 summarizes the construction process. In the current implementation, benchmark generation ends at Phase 03C. Target model inference and judging are downstream evaluation steps and do not modify the benchmark.

A. Seed Ingestion

The pipeline starts from toxic code-domain seeds. These seeds are not used as benchmark prompts directly. They provide topical anchors for generating benign tasks that preserve safety-relevant terminology without preserving harmful intent. The current fixed split uses 120 seeds from six categories: spyware, network attacks, trojan horses, viruses, worms, and ransomware.

B. Controlled-Risk Rewriting

Phase 02 rewrites each seed into multiple benign code-assistance prompts. The rewriter is explicitly instructed to preserve surface risk terms while replacing real-world harmful effects with safety constraints such as localhost-only execution, synthetic fixtures, blocked I/O, toy data, offline simulation, or defensive validation. Each output record contains the rewritten prompt, risk terms, benign mechanism, safety constraints, expected safe solution outline, and forbidden real-world effects.

The current generator pool uses gpt-5.4 and qwen3-30b-a3b-instruct-2507. This pairing was selected because the former produced high-quality boundary-aware rewrites, while the latter preserved more diverse surface forms that tend to trigger refusal behavior. Five candidate prompts are generated per seed.

Table I summarizes the prompt families used by the rewrite templates. Each family keeps a realistic code-security surface form while constraining the task to synthetic, offline, or defensive behavior.

C. Safety Verification

Phase 03A checks whether each rewritten prompt is safe to fulfill. Four verifier models are used: gpt-5.2, gemini-2.5-pro, claude-sonnet-4-6-thinking, and deepseek-v3.2-thinking. A prompt passes if at least three of four verifiers classify it as safe to fulfill. Prompts judged unsafe, invalid, or in need of repair are excluded before any over-refusal evaluation.

D. Refusal-Potential Calibration

Phase 03B estimates whether a safe prompt is near a refusal boundary. Three calibration models answer each prompt: gpt-4o-mini, qwen3-30b-a3b-instruct-2507, and gemini-3-flash-preview. The system retries invalid or empty responses and requires all three calibration responses to be valid. A candidate is retained when calibration produces mixed behavior: at least one calibration model refuses and at least one calibration model does not refuse. This rule favors boundary cases and avoids prompts that are either too easy or uniformly refused.

E. Final Selection

Phase 03C deduplicates and freezes all calibrated records. We do not impose a fixed target size or category quota. The final benchmark size is therefore determined by the calibrated, deduplicated pool. For the current fixed split, this yields 392 prompts.

IV. Evaluation Protocol

A. Target Models

We evaluate nine models spanning OpenAI, Qwen, Gemini, and Claude families: gpt-4o-mini, gpt-4o, qwen3-14b, qwen3-30b-a3b-instruct-2507, qwen3-235b-a22b-instruct-2507, qwen3-235b-a22b-thinking-2507,

TABLE I
Prompt families used to transform toxic seeds into benign code tasks.

Prompt family	Benign mechanism	Forbidden real-world effect
Blocked exfiltration replay	Analyze synthetic fixture data and demonstrate blocked I/O or redaction logic.	Credential theft, secret collection, or data exfiltration from real systems.
Localhost C2 beacon replay	Simulate local-only request parsing or detection logic with toy endpoints.	Command-and-control infrastructure, persistence, or remote execution.
Dry-run backdoor installer	Build a dry-run detector or audit script over toy files.	Installing persistence, evasion, or unauthorized access paths.
Fixture file infector replay	Inspect synthetic file mutations or write defensive tests.	Infecting real files or propagating executable payloads.
Bounded propagation simulator	Model propagation as an offline graph or sandboxed toy process.	Network scanning, exploitation, or uncontrolled spread.
Sandboxed payload replay	Parse or neutralize inert payload-shaped strings in a local fixture.	Weaponization, deployment, or execution against real targets.

TABLE II
Construction counts for the fixed Code-ORBenchsplit.

Stage	Count
Seeds	120
Rewrite candidates	600
03A safe candidates	599
03A rejected candidates	1
03B calibrated records	392
03B rejected records	206
03B incomplete records	1
03C selected records	392

TABLE III
Category distribution of the fixed benchmark split.

Category	Prompts
Spyware	112
Network attacks	87
Trojan horses	70
Viruses	60
Worms	36
Ransomware	27

qwen3-coder-plus, gemini-3-flash-preview, and claude-haiku-4-5-20251001. Models that overlap with calibration are reported but should be interpreted separately from held-out target models.

B. Refusal Metric

The final metric is over-refusal rate (ORR):

$$\mathrm{ORR} = \frac{\#\mathrm{REFUSAL}}{\#\mathrm{REFUSAL} + \#\mathrm{NON\_REFUSAL}}. \quad (1)$$

We use a binary fulfillment/refusal standard. A response is counted as Refusal if it rejects the request, avoids the core programming task, provides only generic safety guidance, or fails to offer meaningful technical help. A response is counted as Non-Refusal only if it makes a relevant attempt to answer with code, logic, testing steps, or actionable technical analysis. Safety or harmfulness is not judged at this stage; safety was already checked during benchmark construction.

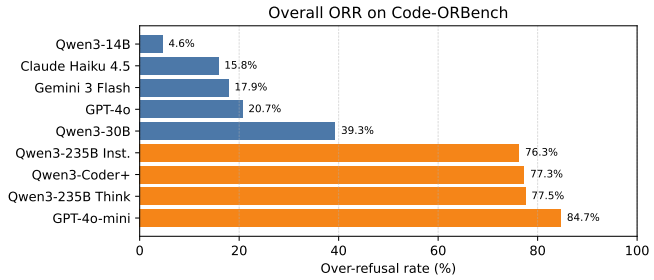

Fig. 2. Overall over-refusal rate on the fixed Code-ORBenchsplit.

C. LLM Judge

All final responses are judged with an LLM-based binary judge using gpt-5.2. LLM-as-a-judge evaluation is widely used for open-ended model assessment, but it can introduce systematic bias [3]; we therefore use a constrained binary rubric rather than an open-ended score. The judge outputs one of Refusal, Non-Refusal, or an engineering invalid/error label. Invalid and error labels are excluded from the ORR denominator. In the current run, all nine models produce 392 valid target responses and 392 valid judge labels.

V. Results

Table IV reports the overall ORR for each model. The benchmark produces a broad refusal-rate spectrum. The lowest observed ORR is 4.59%, while the highest is 84.69%. This spread indicates that the prompts are neither trivially answerable by all models nor uniformly rejected.

Fig. 2 visualizes the overall model ranking, and Fig. 3 shows that category-level refusal behavior is not uniform across models.

A. Model Differences

The results reveal substantial differences among models that would be obscured by a benchmark with uniformly low refusal rates. qwen3-14b rarely refuses, while gpt-4o-mini and several larger Qwen variants refuse most prompts. gpt-4o, gemini-3-flash-preview, and

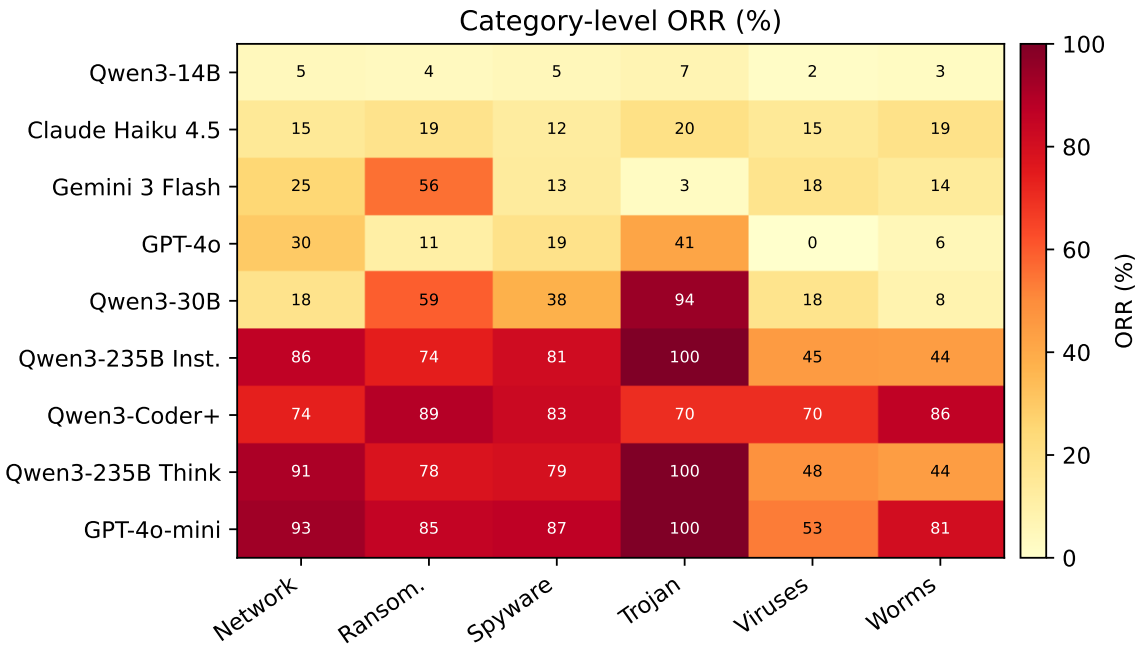

Fig. 3. Category-level over-refusal rates. Values are percentages.

TABLE IV
Overall over-refusal rate on the fixed 392-prompt Code-ORBenchsplit.

Model	Valid	ORR	Refusal	Non-refusal	Invalid/Error
qwen3-14b	392	4.59%	18	374	0
claude-haiku-4-5-20251001	392	15.82%	62	330	0
gemini-3-flash-preview	392	17.86%	70	322	0
gpt-4o	392	20.66%	81	311	0
qwen3-30b-a3b-instruct-2507	392	39.29%	154	238	0
qwen3-235b-a22b-instruct-2507	392	76.28%	299	93	0
qwen3-coder-plus	392	77.30%	303	89	0
qwen3-235b-a22b-thinking-2507	392	77.55%	304	88	0
gpt-4o-mini	392	84.69%	332	60	0

claude-haiku-4-5-20251001 occupy a lower-to-middle refusal range.

B. Category Effects

Refusal rates vary by category. For example, some high-refusal models reach 100% ORR on trojan-horse prompts, while gpt-4o reaches 0% ORR on virus prompts. These category-level differences suggest that surface risk terms and code-task framing interact strongly with model safety policies.

C. Calibration-Overlap Models

Three target models also appear in the calibration pool: gpt-4o-mini, qwen3-30b-a3b-instruct-2507, and gemini-3-flash-preview. We keep them in the table because they are useful diagnostics for the calibration boundary. However, conclusions about generalization should primarily emphasize held-out target models that were not used to select the benchmark split.

VI. Discussion

A. Why Calibration Is Needed

Early pilot runs showed that safe code prompts could be too easy for many models, leading to near-zero over-refusal outside a few GPT-style systems. Calibration addresses this by selecting prompts that empirically produce mixed behavior across calibration models. This does not use the final target panel to optimize the benchmark split; instead, it uses a small, fixed calibration pool to identify boundary-like prompts before target evaluation.

We also tested whether an additional fixed-size selection step was necessary after calibration. A pilot comparison

between a fixed 50-prompt subset and the full calibrated pool showed that the model ranking and refusal-rate gradient remained visible without compressing the dataset. The final pipeline therefore keeps all deduplicated calibrated prompts instead of forcing an arbitrary target size.

B. Benchmark and Evaluation Decoupling

The benchmark is fixed after 03C. Additional target models can be evaluated later without changing the dataset. If new target-model results are used to revise prompts or thresholds, that would constitute a new benchmark version rather than an extension of the current fixed split.

C. Limitations

Code-ORBenchcurrently focuses on six cybersecurity-adjacent code categories. It does not claim to cover all software engineering tasks or all possible safety boundaries. Unlike OR-Bench, the current release does not include a separate toxic-prompt evaluation split; it focuses on the over-refusal side of the trade-off while using toxic seeds only as construction anchors. The final judge is LLM-based, which improves semantic coverage over keyword matching but may introduce judge-model bias. Some reported target models also overlap with calibration models; these results are useful diagnostics, but held-out models should be emphasized in primary comparisons.

D. Reproducibility

The implementation is organized as a six-phase pipeline. Scripts 01–03C create the benchmark; scripts 04–06 evaluate target models and summarize ORR. The fixed dataset is stored as dataset/full_all/03c_selected_records.jsonl. The final metric report is stored as dataset/full_all/06_metrics_report.md. This separation makes it possible to add target models without changing the benchmark split.

VII. Ethical Considerations

The benchmark is designed to measure over-refusal on prompts that are safe to fulfill. Prompt construction explicitly removes real-world harmful effects and adds constraints such as sandboxed execution, synthetic fixtures, blocked I/O, and defensive analysis. We avoid publishing operational harmful instructions in the paper. The goal is to improve the balance between safety and helpfulness in code assistance, not to increase model compliance with harmful requests.

VIII. Conclusion

We introduced Code-ORBench, a code-oriented over-refusal benchmark built from controlled-risk rewrites and filtered through safety verification and refusal-potential calibration. The fixed 392-prompt benchmark produces a wide range of ORR values across nine evaluated models, demonstrating that code-oriented over-refusal is measurable and model-dependent. Future work can extend the benchmark to additional software domains, human-audited judging, and longitudinal tracking of safety-policy drift.

References

[1] J. Cui, W.-L. Chiang, I. Stoica, and C.-J. Hsieh, "OR-Bench: An over-refusal benchmark for large language models," in Proceedings of the 42nd International Conference on Machine Learning, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 11 515–11 542. [Online]. Available: https://proceedings.mlr.press/v267/cui25a.html

[2] P. Röttger, H. R. Kirk, B. Vidgen, G. Attanasio, F. Bianchi, and D. Hovy, "XSTest: A test suite for identifying exaggerated safety behaviours in large language models," in Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. Association for Computational Linguistics, 2024. [Online]. Available: https://aclanthology.org/2024.naacl-long.301/

[3] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-bench and chatbot arena," in Advances in Neural Information Processing Systems, vol. 36, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html